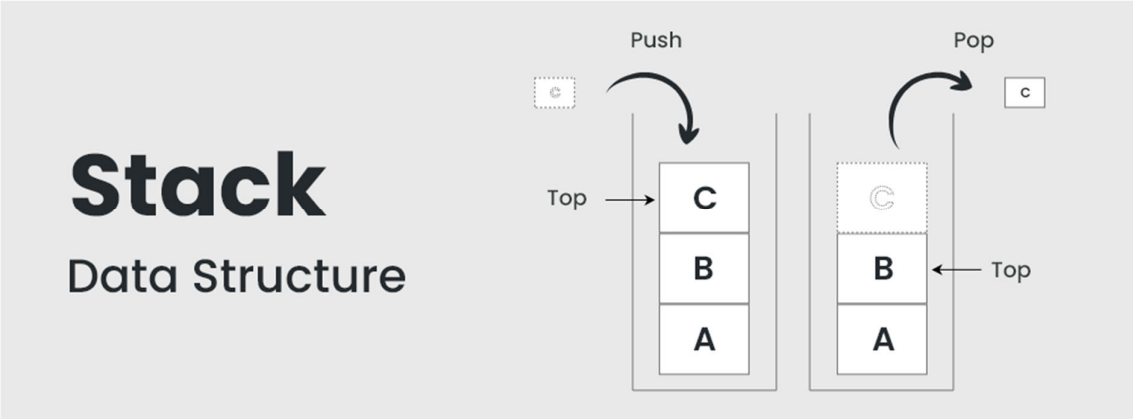
Data Structure Using C

Unit IV

STACKS, QUEUES AND RECURSION

Stack

Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO(Last In First Out) or FILO(First In Last Out). LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last.



There are many real-life examples of a stack. Consider an example of plates stacked over one another in the canteen. The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time. So, it can be simply seen to follow LIFO(Last In First Out)/FILO(First In Last Out) order.

Standard Stack Operations

The following are some common operations implemented on the stack:

push(): When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.

pop(): When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.

isEmpty(): It determines whether the stack is empty or not.

isFull(): It determines whether the stack is full or not.

top(): It returns the top element of the stack.

size(): It returns the total number of elements available in a stack.

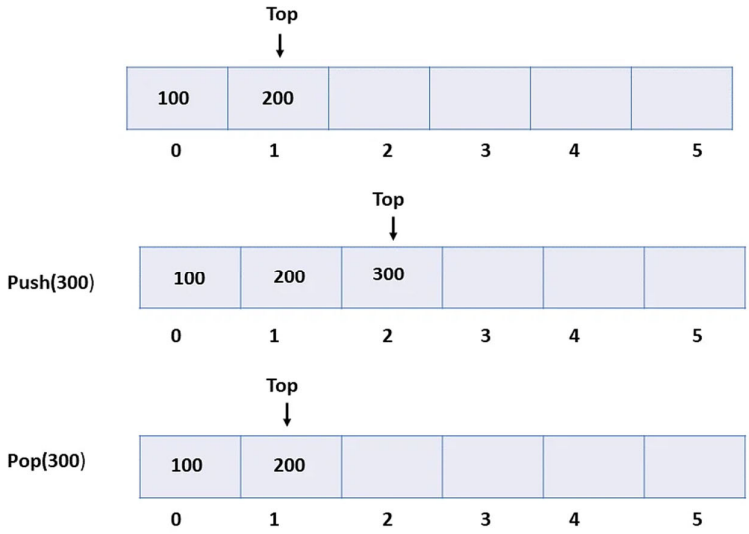
change(): It changes the element at the given position.

display(): It prints all the elements available in the stack.

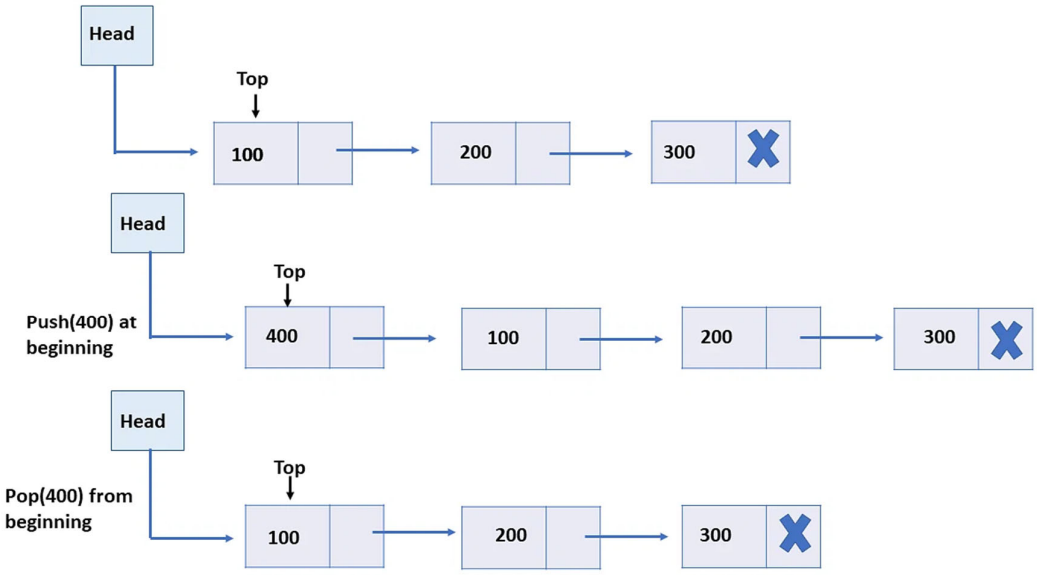
Implementation of STACK in Data Structure

You can perform the implementation of stacks in data structures using two data structures that are an array and a linked list.

Array: In array implementation, the stack is formed using an array. All the operations are performed using arrays. You will see how all operations can be implemented on the stack in data structures using an array data structure.



Linked-List: Every new element is inserted as a top element in the linked list implementation of stacks in data structures. That means every newly inserted element is pointed to the top. Whenever you want to remove an element from the stack, remove the node indicated by the top, by moving the top to its previous node in the list.



Applications of Stack

- Balancing of symbols
- String reversal
- UNDO/REDO
- Recursion
- DFS (Depth First Search)
- Backtracking
- Expression conversion
- Expression evaluation
- Memory management

Balancing of symbols

Iru#wdflqj#wkh#dojruwkp #dw#kv#dvvxp h#wkdw#wkh#qsxw#v#,#+,',#

Input Symbol	Operation	Stack	Output
(	Push (	(	
)	Pop (Test if (and A[i] match? YES		
(	Push (	(	
(	Push (	((	
)	Pop (Test if (and A[i] match? YES	(	
[	Push [	([	
(	Push (	([(	
)	Pop) Test if (and A[i] match? YES	([	
]	Pop [Test if [and A[i] match? YES	(	
)	Pop (Test if (and A[i] match? YES		
	Test if Stack is Empty? YES		TRUE

Expression Conversion

Precedence Order and Associativity of Operators

The precedence order of operators is given in the below table.

Precedence	Type	Operators	Associativity
1	Postfix	() [] -> . ++ —	Left to Right
2	Unary	+ – ! ~ ++ — (type)* & sizeof	Right to Left
3	Multiplicative	* / %	Left to Right
4	Additive	+ –	Left to Right
5	Shift	<<, >>	Left to Right
6	Relational	< <= > >=	Left to Right
7	Equality	== !=	Left to Right
8	Bitwise AND	&	Left to Right
9	Bitwise XOR	^	Left to Right
10	Bitwise OR	|	Left to Right
11	Logical AND	&&	Left to Right
12	Logical OR	||	Left to Right
13	Conditional	?:	Right to Left
14	Assignment	= += -+ *= /= %= >>= <<= &= ^= |=	Right to Left
15	Comma	,	Left to Right

Expression − It is a legal combination of operands and operations.

Types of expressions

There are three types of expressions in C language on which the conversions and evaluations can be carried out. They are explained below −

Infix Notation expression − Operator is in between the operands. For example, A+B

Prefix (Polish) Notation expression − Operator is before the operands. For example, +AB

Postfix (Reverse Polish) Notation expression − Operator is after the operands. For example, AB+

Conversion of infix to postfix and infix to prefix is explained below −

Infix to postfix	**Infix to prefix**
A+ B*C	A+ B*C
A+ BC *	A+ *BC
ABC* +	+A*BC

For example,

Convert A+B*C / D-E+F infix to postfix and prefix.

Infix to prefix	Infix to postfix
A +B*C / D-E+F	A +B*C / D-E+F
A +*BC / D-E+F	A +BC* / D-E+F
A + / *BCD -E+F	A +BC*D /-E+F
+A /*BCD -E +F	ABC *D /+ -E+F
-+A/*BCDE +F	ABC *D/ +E- +F
+ - +A/*BCDEF	ABC *D / +E –F+

Infix to Postfix (or Infix to reverse-Polish notation)

Algorithm for Conversion of Infix to Postfix

- Scan all the symbols one by one from left to right in the given Infix Expression.
- If the reading symbol is an operand, then immediately append it to the Postfix Expression.
- If the reading symbol is left parenthesis '(', then Push it onto the Stack.
- If the reading symbol is right parenthesis ')', then Pop all the contents of the stack until the respective left parenthesis is popped and append each popped symbol to Postfix Expression.
- If the reading symbol is an operator (+, –, *, /), then Push it onto the Stack. However, first, pop the operators which are already on the stack that have higher or equal precedence than the current operator and append them to the postfix. If an open parenthesis is there on top of the stack then push the operator into the stack.
- If the input is over, pop all the remaining symbols from the stack and append them to the postfix.

E.g. 1

Input: A * B + C / D

Output: AB*CD/+

E.g. 2

Input: A*(B+C)/D

Output: ABC+*D/

E.g.3

K + L - M*N + (O^P) * W/U/V * T + Q

Input Expression	Stack	Postfix Expression
K		K
+	+	
L	+	K L
-	-	K L+
M	-	K L+ M
*	- *	K L+ M
N	- *	K L + M N

+	+	K L + M N* K L + M N* -
(	+ (	K L + M N *-
O	+ (	K L + M N * - O
^	+ (^	K L + M N* - O
P	+ (^	K L + M N* - O P
)	+	K L + M N* - O P ^
*	+ *	K L + M N* - O P ^
W	+ *	K L + M N* - O P ^ W
/	+ /	K L + M N* - O P ^ W *
U	+ /	K L + M N* - O P ^W*U
/	+ /	K L + M N* - O P ^W*U/
V	+ /	KL + MN*-OP^W*U/V
*	+ *	KL+MN*-OP^W*U/V/
T	+ *	KL+MN*-OP^W*U/V/T
+	+	KL+MN*-OP^W*U/V/T* KL+MN*-OP^W*U/V/T*+
Q	+	KL+MN*-OP^W*U/V/T*Q
		KL+MN*-OP^W*U/V/T*+Q+

Infix to prefix (or Infix to polish notation)

Algorithm

To convert an infix expression to a prefix expression, we can use the stack data structure. The idea is as follows:

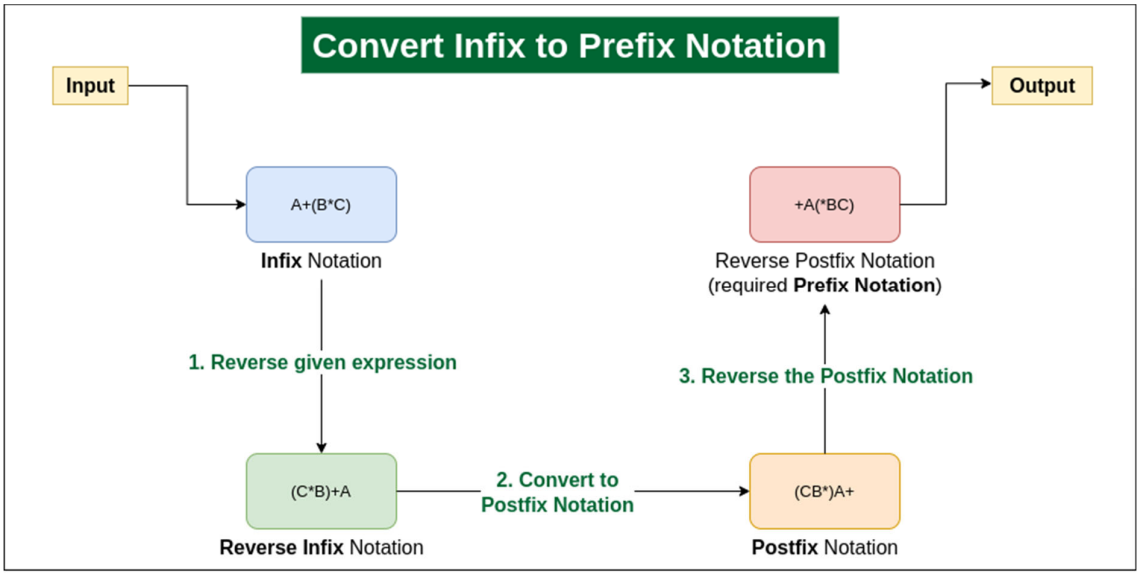
Step 1: Reverse the infix expression. Note while reversing each ‘(‘ will become ‘)’ and each ‘)’ becomes ‘(‘.

Step 2: Convert the reversed infix expression to “nearly” postfix expression.

While converting to postfix expression, instead of using pop operation to pop operators with greater than or equal precedence, here we will only pop the operators from stack that have greater precedence.

Step 3: Reverse the postfix expression.

The stack is used to convert infix expression to postfix form.



Input: A * B + C / D

Output: + * A B/ C D

K + L - M * N + (O^P) * W/U/V * T + Q

If we are converting the expression from infix to prefix, we need first to reverse the expression.

The Reverse expression would be:

Q + T * V/U/W * (P^O) + N*M - L + K

To obtain the prefix expression, we have created a table that consists of three columns, i.e., input expression, stack, and prefix expression. When we encounter any symbol, we simply add it into the prefix expression. If we encounter the operator, we will push it into the stack.

Input expression	Stack	Prefix expression
Q		Q
+	+	Q
T	+	QT
*****	+*	QT
V	+*	QTV

/	+*/	QTV
U	+*/	QTVU
/	+*//	QTVU
W	+*//	QTVUW
*****	+*//*	QTVUW
)	+*//*)	QTVUW
P	+*//*)	QTVUWP
^	+*//*)^	QTVUWP
O	+*//*)^	QTVUWPO
(	+*//*	QTVUWPO^
+	++	QTVUWPO^*//*
N	++	QTVUWPO^*//*N
*****	++*	QTVUWPO^*//*N
M	++*	QTVUWPO^*//*NM
-	++-	QTVUWPO^*//*NM*
L	++-	QTVUWPO^*//*NM*L
+	++-+	QTVUWPO^*//*NM*L
K	++-+	QTVUWPO^*//*NM*LK
		QTVUWPO^*//*NM*LK+-++

The above expression, i.e., QTVUWPO^*//*NM*LK+-++, is not a final expression. We need to reverse this expression to obtain the prefix expression.

Output: ++-+KL*MN*//*^OPWVTQ

Convert Postfix to Infix Expression

Algorithm:

Iterate the given expression from left to right, one character at a time

1. If a character is operand, push it to stack.
2. If a character is an operator,
    1. pop operand from the stack, say it's s1.
    2. pop operand from the stack, say it's s2.
    3. perform (s2 operator s1) and push it to stack.
3. Once the expression iteration is completed, initialize the result string and pop out from the stack and add it to the result.
4. Return the result.

Postfix Expression : ABC/-AK/L-*			
Token	**Action**	**Stack**	**Notes**
A	Push **A** to stack	[A]	
B	Push **B** to stack	[A, B]	
C	Push **C** to stack	[A, B, C]	
/	Pop **C** from stack	[A, B]	Pop two operands from stack, C and B. Perform B/C and push (B/C) to stack
	Pop **B** from stack	[A]	
	Push **(B/C)** to stack	[A, (B/C)]	
-	Pop **(B/C)** from stack	[A]	Pop two operands from stack, (B/C) and A. Perform A-(B/C) and push (A-(B/C)) to stack
	Pop **A** from stack	[]	
	Push **(A-(B/C))** to stack	[((A-(B/C))]	
A	Push **A** to stack	[(A-(B/C)), A]	
K	Push **K** to stack	[(A-(B/C)), A, K]	
/	Pop **K** from stack	[(A-(B/C)), A]	Pop two operands from stack, K and A. Perform A/K and push (A/K) to stack
	Pop **A** from stack	[((A-(B/C))]	
	Push **(A/K)** to stack	[(A-(B/C)), (A/K)]	
L	Push **L** to stack	[(A-(B/C)), (A/K), L]	
-	Pop **L** from stack	[(A-(B/C)), (A/K)]	Pop two operands from stack, L and (A/K). Perform (A/K)-L and push ((A/K)-L) to stack
	Pop **(A/K)** from stack	[((A-(B/C))]	
	Push **((A/K)-L)** to stack	[(A-(B/C)), ((A/K)-L)]	
*	Pop **((A/K)-L)** from stack	[((A-(B/C))]	Pop two operands from stack, (A/K)-L) and A-(B/C). Perform (A-(B/C))*((A/K)-L) and push ((A-(B/C))*((A/K)-L)) to stack
	Pop **((A-(B/C))** from stack	[]	
	Push **((A-(B/C))*((A/K)-L))** to stack	[((A-(B/C))*((A/K)-L))]	
Infix Expression: ((A-(B/C))*((A/K)-L))			

Algorithm for Prefix to Infix Conversion

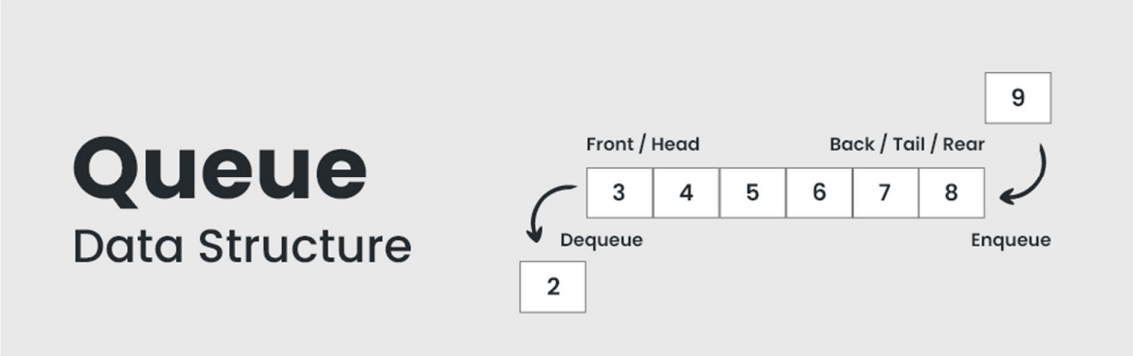
1. First, we will read the given prefix expression in reverse order, i.e., from right to left.
2. We will use a stack to store the operands. So, if we encounter a symbol, we will push it into the stack
3. If we encounter an operator, then we will pop two operands out of the stack.
4. At last, we will create a concatenated Python string of the two operands and put the operator between them.
5. infix = (operand_1 + operator + operand_2)
6. Then we will push the infix string back into the stack.
7. We will repeat the above steps until we reach the end of the given prefix expression.
8. At the end of the execution, we will be left with a single string of the infix expression.

Introduction to Queue

What is Queue Data Structure?

A Queue is defined as a linear data structure that is open at both ends and the operations are performed in First In First Out (FIFO) order.

We define a queue to be a list in which all additions to the list are made at one end, and all deletions from the list are made at the other end. The element which is first pushed into the order, the operation is first performed on that.



FIFO Principle of Queue:

- A Queue is like a line waiting to purchase tickets, where the first person in line is the first person served. (i.e. First come first serve).
- Position of the entry in a queue ready to be served, that is, the first entry that will be removed from the queue, is called the front of the queue(sometimes, head of the queue), similarly, the position of the last entry in the queue, that is, the one most recently added, is called the rear (or the tail) of the queue. See the below figure.



Queue Data Structure

Characteristics of Queue:

- Queue can handle multiple data.
- We can access both ends.
- They are fast and flexible.

Basic Operations for Queue in Data Structure

Unlike arrays and linked lists, elements in the queue cannot be operated from their respective locations. They can only be operated at two data pointers, front and rear. Also, these operations involve standard procedures like initializing or defining data structure, utilizing it, and then wholly erasing it from memory. Here, you must try to comprehend the operations associated with queues:

Enqueue() - Insertion of elements to the queue.

Dequeue() - Removal of elements from the queue.

Peek() - Acquires the data element available at the front node of the queue without deleting it.

isFull() - Validates if the queue is full.

isNull() - Checks if the queue is empty.

Implementation of Queue

A queue can be implemented in two ways:

Sequential allocation: It can be implemented using an array. A queue implemented using an array can organize only a limited number of elements.

Linked list allocation: It can be implemented using a linked list. A queue implemented using a linked list can organize an unlimited number of elements.

Applications of Queue

Printers: Queue data structure is used in printers to maintain the order of pages while printing.

Interrupt handling in computes: The interrupts are operated in the same order as they arrive, i.e., interrupt which comes first, will be dealt with first.

Process scheduling in Operating systems: Queues are used to implement round-robin scheduling algorithms in computer systems.

Switches and Routers: Both switch and router interfaces maintain ingress (inbound) and egress (outbound) queues to store packets.

Customer service systems: It develops call center phone systems using the concepts of queues.

Types of Queues:

Simple Queue: Simple queue also known as a linear queue is the most basic version of a queue. Here, insertion of an element i.e. the Enqueue operation takes place at the rear end and removal of an element i.e. the Dequeue operation takes place at the front end. Here problem is that if we pop some item from front and then rear reach to the capacity of the queue and although there are empty spaces before front means the queue is not full but as per condition in isFull() function, it will show that the queue is full then. To solve this problem we use circular queue.

Circular Queue: In a circular queue, the element of the queue act as a circular ring. The working of a circular queue is similar to the linear queue except for the fact that the last element is connected to the first element. Its advantage is that the memory is utilized in a better way. This is because if there is an empty space i.e. if no element is present at a certain position in the queue, then an element can be easily added at that position using modulo capacity(%n).

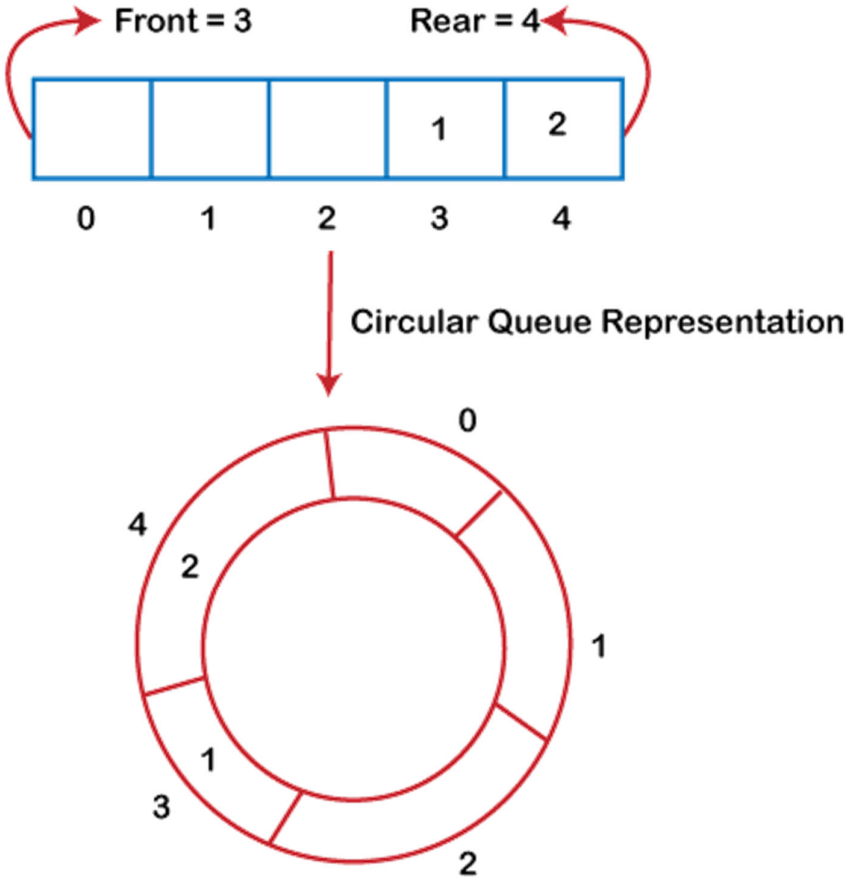
Priority Queue: This queue is a special type of queue. Its specialty is that it arranges the elements in a queue based on some priority. The priority can be something where the element with the highest value has the priority so it creates a queue with decreasing order of values. The priority can also be such that the element with the lowest value gets the highest priority so in turn it creates a queue with increasing order of values. In pre-define priority queue, C++ gives priority to highest value whereas Java gives priority to lowest value.

Double Ended Queue (Deque): As the name suggests double ended, it means that an element can be inserted or removed from both ends of the queue, unlike the other queues in which it can be done only from one end. Because of this property, it may not obey the First In First Out property.

Circular Queue

Why was the concept of the circular queue introduced?

There was one limitation in the array implementation of Queue. If the rear reaches to the end position of the Queue then there might be possibility that some vacant spaces are left in the beginning which cannot be utilized. So, to overcome such limitations, the concept of the circular queue was introduced.



As we can see in the above image, the rear is at the last position of the Queue and front is pointing somewhere rather than the 0th position. In the above array, there are only two elements and other three positions are empty. The rear is at the last position of the Queue; if we try to insert the element then it will show that there are no empty spaces in the Queue. There is one solution to avoid such wastage of memory space by shifting both the elements at the left and adjust the front and rear end accordingly. It is not a practically good approach because shifting all the elements will consume lots of time. The efficient approach to avoid the wastage of the memory is to use the circular queue data structure.

What is a Circular Queue?

A circular queue is similar to a linear queue as it is also based on the FIFO (First In First Out) principle except that the last position is connected to the first position in a circular queue that forms a circle. It is also known as a Ring Buffer.

Operations on Circular Queue

The following are the operations that can be performed on a circular queue:

Front: It is used to get the front element from the Queue.

Rear: It is used to get the rear element from the Queue.

enQueue(value): This function is used to insert the new value in the Queue. The new element is always inserted from the rear end.

deQueue(): This function deletes an element from the Queue. The deletion in a Queue always takes place from the front end.

Applications of Circular Queue

The circular Queue can be used in the following scenarios:

Memory management: The circular queue provides memory management. As we have already seen that in linear queue, the memory is not managed very efficiently. But in case of a circular queue, the memory is managed efficiently by placing the elements in a location which is unused.

CPU Scheduling: The operating system also uses the circular queue to insert the processes and then execute them.

Traffic system: In a computer-control traffic system, traffic light is one of the best examples of the circular queue. Each light of traffic light gets ON one by one after every jinterval of time. Like red light gets ON for one minute then yellow light for one minute and then green light. After green light, the red light gets ON.

Deque (or double-ended queue)

What is a Deque (or double-ended queue)?

The deque stands for Double Ended Queue. Deque is a linear data structure where the insertion and deletion operations are performed from both ends. We can say that deque is a generalized version of the queue.

Though the insertion and deletion in a deque can be performed on both ends, it does not follow the FIFO rule. The representation of a deque is given as follows –



Representation of deque

Types of deque

There are two types of deque -

- Input restricted queue
- Output restricted queue

Input restricted Queue

In input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



input restricted double ended queue

Output restricted Queue

In output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.



Output restricted double ended queue

Applications of deque

1. Deque can be used as both stack and queue, as it supports both operations.
2. Deque can be used as a palindrome checker means that if we read the string from both ends, the string would be the same.

Recursion

What is Recursion in C?

Recursion is the process of calling a function itself repeatedly until a particular condition is met. A function that calls itself directly or indirectly is called a recursive function and such kind of function calls are called recursive calls.

In C, recursion is used to solve a complex problem. Using recursion we can solve a complex problem in a small piece of code by breaking down the problem. We can solve large numbers of problems using recursion for example factorial of a number, generating Fibonacci series, generating subsets etc.

Recursive Functions

A function that calls a copy of itself is called Recursive Function. The recursive functions contain a call to themselves somewhere in the function body. Moreover, such functions can contain multiple recursive calls

Basic Structure of Recursive Functions

The basic syntax structure of the recursive functions is:

```
type function_name (args) {
   // function statements
   // base condition
   // recursion case (recursive call)
}
```

Fundamentals of Recursion in C

The fundamental of recursion consists of two objects which are essential parts of any recursive function. These are:

- Recursion Case
- Base Condition

1. Recursion Case

The recursion case refers to the recursive call present in the recursive function. It decides what type of recursion will occur and the problem will be divided into smaller subproblems.

The recursion case defined in the nSum() function of the above example is:

int res = n + nSum(n - 1);

The recursion case if often represented mathematically is a recurrence relation.

f(N) = N + f(N - 1);

This recurrence relation is used for the complexity analysis of the method.

2. Base Condition

The base condition specifies when the recursion is going to terminate. It is the condition that determines the exit point of the recursion. We have to be careful while defining the base condition because the incorrect or incomplete base condition may lead the recursion to run for infinite times.

Note: *It is important to define the base condition before the recursive case otherwise, the base condition may never encountered and recursion might continue till infinity.*

Types of Recursion

The recursion can be classified into different types based on what kind of recursive case is present in the recursion.

1. **Direct Recursion**
   - Head Recursion
   - Tail Recursion
   - Tree Recursion
2. **Indirect Recursion**

1. Direct Recursion

Direct recursion is the most common type of recursion, where a function calls itself directly within its own body. The recursive call can occur once or multiple times within the function due to which we can further classify the direct recursion

A. Head Recursion

The head recursion is a linear recursion where the position of its one and only recursive call is at the start of the function. It is generally the first statement in the function.

B. Tail Recursion

The tail recursion is also a liner recursion like head recursion but the position of the recursive call is at the end of the function. Due to this, the tail recursion can be optimized to minimize the stack memory usage. This process is called Tail Call Optimization.

In the first example, the nSum() does the tail recursion.

C. Tree Recursion

In tree recursion, there are multiple recursive calls present in the body of the function. Due to this, while tracing the program flow, it makes a tree-like structure, hence the name Tree Recursion.

2. Indirect Recursion

Indirect recursion is an interesting form of recursion where a function calls another function, which eventually calls the first function or any other function in the chain, leading to a cycle of function calls. In other words, the functions are mutually recursive. This type of recursion involves multiple functions collaborating to solve a problem.

Stack Overflow

The program's call stack has limited memory assign to it by the operating system and it is generally enough for program execution. But if we have a recursive function that goes on for infinite times, sooner or later, the memory will be exhausted and no more data can be stored. This is called stack overflow. In other words,

Stack overflow is the error that occurs occurs when the call stack of the program cannot store more data resulting in program termination.

It is one of the most common errors that is associated with the recursion.